Resumo:
Complexidade de Algoritmos

SQUAD 11

Taxa de Crescimento Assintótico

A análise assintótica afasta a intuição vaga e traz previsibilidade científica. Avaliamos a eficiência pela taxa de crescimento quando o tamanho da entrada tende ao infinito.

$$\Theta(1) < \Theta(\log n) < \Theta(n) < \Theta(n \log n) < \Theta(n^2) < \Theta(n^3) < \Theta(2^n)$$

Regra de Dominância: Constantes multiplicativas e termos de menor ordem são desprezados na escala de longo prazo.

Ordens Assintóticas e Limites



Notação Big-O (O)

Define o limite superior assintótico. Representa o tempo máximo garantido de execução.

$$f(n) \leq c \cdot g(n)$$



Notação Theta (Θ)

Define o limite estrito ou firme, operando como um "sanduíche" funcional para garantir exatidão.

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

Notação Omega (Ω)

Define o limite inferior assintótico. Mostra que o algoritmo requerirá pelo menos este tempo.

$$c \cdot g(n) \leq f(n)$$

Questão 1

SITUAÇÃO-PROBLEMA

No projeto de algoritmos e estruturas de dados de alta performance, a compreensão das dimensões quantitativas e dos limites teóricos de complexidade é indispensável para subsidiar decisões técnicas maduras de desenvolvimento de software. Um engenheiro de software precisa projetar um algoritmo para processar grandes volumes de dados corporativos de suporte de tamanho n. Para isso, ele modela analiticamente dois algoritmos concorrentes e obtém duas funções de custo temporal: $f(n) = n^2$ e $g(n) = n \log_2 n$. Ao avaliar a eficiência de longo prazo das soluções sob picos críticos de processamento, o engenheiro deve classificar esses algoritmos em categorias de tempo utilizando as definições formais de limites assintóticos fornecidas pela Notação Big-O.

DADOS DE SUPORTE

De acordo com Cormen et al. (2012), a Notação Big-O estabelece uma ferramenta analítica fundamental para classificar o crescimento relativo de funções matemáticas. Do ponto de vista conceitual, dizemos que $f(n) = O(g(n))$ se existirem constantes reais positivas c e n_0 tais que $0 \leq f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$. Esse formalismo é essencial para que o estudante de computação abandone intuições vagas ou estimativas superficiais e busque rigor e precisão científica em suas suposições de engenharia de software.

Considere as duas asserções a seguir e a relação proposta entre elas:

I. A função de complexidade temporal quadrática $f(n) = n^2$ é limitada assintoticamente de forma superior pela classe $O(n \log_2 n)$, ou seja, $f(n) = O(n \log_2 n)$,

PORQUE

II. para valores suficientemente grandes de n, a função de crescimento linearítmico $g(n) = n \log_2 n$ domina de maneira estrita a função quadrática $f(n) = n^2$ à medida que o tamanho da entrada cresce em direção ao infinito, satisfazendo a desigualdade $n^2 \leq c \cdot (n \log_2 n)$ para alguma constante real positiva c.

COMANDO DA QUESTÃO

A respeito dessas asserções, assinale a opção correta:

ALTERNATIVAS DE RESPOSTA

A) As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.

B) As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.

C) A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.

D) As asserções I e II são proposições falsas.

Modelo de Contagem de Knuth

Quando a abstração assintótica falha em resolver empates de eficiência, o Modelo de Knuth fornece o cálculo exato de instruções críticas no nível da máquina.

$$T(n) = \sum c_i \cdot t_i$$

- **Estudo de Caso:** Busca Sequencial com Sentinela.
- A inserção da chave na última posição do vetor elimina a verificação de limite do laço principal.
- Reduz operações lógicas, garantindo tempo ótimo de constante **Θ(1)** no melhor caso e linear **Θ(n)** no pior.



Questão 2

SITUAÇÃO-PROBLEMA

Durante o desenvolvimento de um componente de busca e recuperação de registros para um SGBD de alta performance, uma equipe de desenvolvimento deparou-se com a necessidade de prever com precisão científica o consumo de tempo de execução de uma rotina iterativa crítica sob grandes volumes de dados. Para embasar decisões técnicas de desenvolvimento e garantir a previsibilidade operacional de tempo de resposta, o engenheiro principal aplicou o Modelo de Knuth para traduzir o algoritmo em um modelo matemático de contagem detalhada de instruções críticas, recusando-se a confiar em intuições vagas ou em estimativas superficiais comuns no desenvolvimento de software.

DADOS DE SUPORTE

De acordo com Knuth (1997) e Cormen et al. (2012), a modelagem formal de contagem de operações consiste em associar um custo de execução unitário c_i a cada linha de instrução i e identificar a frequência exata de execução t_i de cada uma delas em função do tamanho da entrada n.

Considere o algoritmo clássico de busca sequencial com sentinela implementado em linguagem C, em que o vetor v possui n elementos de dados de suporte e a posição $v[n-1]$ está reservada para armazenar temporariamente a sentinela:

```
1  int busca_sentinela(int v[], int n, int chave) {
2      int i = 0;
3      v[n - 1] = chave;          // Linha 1 (custo c_1)
4      while (v[i] != chave) {    // Linha £ (custo c_£)
5          i++;                   // Linha 3 (custo c_3)
6      }
7      if (i < n - 1) {           // Linha 4 (custo c_4)
8          return i;              // Linha 5 (custo c_5)
9      }
10     return -1;                 // Linha 6 (custo c_6)
11 }
```

Considere os custos de tempo c_1 a c_6 associados a cada linha i do código, respectivamente. Analise as seguintes afirmativas sobre a contagem exata e o crescimento assintótico desse algoritmo:

I. No pior caso de busca (quando a chave não está originalmente no vetor v, exceto na sentinela), a Linha 2 (teste do loop) executa exatamente n vezes com custo de tempo de execução correspondente a $n \cdot c_2$.

II. A função de complexidade de tempo de pior caso exata desse algoritmo é expressa analiticamente por $T_{pior}(n) = c_1 + n \cdot c_2 + (n-1) \cdot c_3 + c_4 + c_6$, o que o classifica de forma correta na ordem de complexidade linear $\Theta(n)$.

III. No melhor caso de busca (quando a chave é encontrada imediatamente na primeira posição v), o laço `while` executa apenas uma única comparação, resultando em tempo de execução constante de classe assintótica $\Theta(1)$.

COMANDO DA QUESTÃO

Considerando o Modelo de Knuth para a modelagem formal de contagem de instruções, as propriedades matemáticas de progressões e somatórios, a Notação Big-O para classes de complexidade e os limites teóricos apresentados, é correto o que se afirma em:

ALTERNATIVAS DE RESPOSTA

A) I e II, apenas.

B) I e III, apenas.

C) II e III, apenas.

D) I, II e III.

Variação de Comportamento Dinâmico



Pior Caso

Define o limite superior de tempo. Garante que o algoritmo não excederá os recursos calculados. Vital para SLAs e sistemas de tempo real críticos.



Caso Médio

Comportamento estatístico sob todas as entradas possíveis (distribuição uniforme). Reflete o desempenho cotidiano e prático do sistema.



Melhor Caso

O cenário mais favorável possível. É extremamente útil para validar otimizações lógicas e o desenvolvimento de caminhos rápidos (fast-paths).

Questão 3

SITUAÇÃO-PROBLEMA

No desenvolvimento de sistemas de software crítico de alto desempenho — como motores de bancos de dados relacionais e indexadores de busca *web* —, a escolha adequada de estruturas lógicas de armazenamento e processamento de dados deve ser apoiada por análises teóricas rigorosas, recusando-se a confiar em estimativas superficiais ou intuições vagas. Durante a otimização de um microsserviço de análise de dados estruturados de larga escala, um cientista da computação precisa classificar e ordenar cinco algoritmos concorrentes em termos de eficiência de pior caso. O engenheiro aplica o Modelo de Knuth para modelar analiticamente a contagem exata das instruções críticas dominantes, obtendo cinco funções de custo de tempo de execução denominadas de $T_I(n)$ a $T_V(n)$.

DADOS DE SUPORTE

De acordo com Cormen et al. (2012), a modelagem matemática das classes de complexidade representada pela Notação Big-O permite abstrair o comportamento das funções de custo sob picos de carregamento de dados e prever formalmente os limites de tempo de execução quando o tamanho da entrada n cresce de forma ilimitada para o infinito.

Considere as cinco funções de contagem de operações dominantes obtidas analiticamente para as instruções críticas de cada algoritmo, operando sob uma entrada de tamanho n (onde $n \geq 2$):

- **Algoritmo I:** $T_I(n) = 100n + 15$
- **Algoritmo II:** $T_{II}(n) = \log_2(n^5)$
- **Algoritmo III:** $T_{III}(n) = \sum_{i=1}^{n} i$
- **Algoritmo IV:** $T_{IV}(n) = n \log_2(n^3)$
- **Algoritmo V:** $T_V(n) = 2^n + n^4$

Para fins de ordenação e seleção de projeto, os algoritmos devem ser listados em **ordem crescente** da taxa de crescimento de suas funções de complexidade assintótica de tempo de pior caso, ou seja, partindo do algoritmo de **maior eficiência** (menor taxa de crescimento) para o de **menor eficiência** (maior taxa de crescimento).

COMANDO DA QUESTÃO

A partir da simplificação analítica de cada modelo de contagem estruturado e aplicando as definições formais de limites assintóticos da Notação Big-O, assinale a alternativa que apresenta a ordenação correta das funções de custo de tempo, do algoritmo mais eficiente ao menos eficiente:

ALTERNATIVAS DE RESPOSTA

A) II – I – IV – III – V, representando as classes de complexidade $\Theta(\log n)$, $\Theta(n)$, $\Theta(n \log n)$, $\Theta(n^2)$ e $\Theta(2^n)$, do mais eficiente ao menos eficiente.

B) II – IV – I – III – V, representando as classes de complexidade $\Theta(\log n)$, $\Theta(n \log n)$, $\Theta(n)$, $\Theta(n^2)$ e $\Theta(2^n)$, do mais eficiente ao menos eficiente.

C) I – II – IV – III – V, representando as classes de complexidade $\Theta(n)$, $\Theta(\log n)$, $\Theta(n \log n)$, $\Theta(n^2)$ e $\Theta(2^n)$, do mais eficiente ao menos eficiente.

D) II – I – III – IV – V, representando as classes de complexidade $\Theta(\log n)$, $\Theta(n)$, $\Theta(n^2)$, $\Theta(n \log n)$ e $\Theta(2^n)$, do mais eficiente ao menos eficiente.

Modelagem de Laços Iterativos

Laços Triangularmente Aninhados

O laço interno depende dinamicamente do laço externo. Solucionado via Soma de Gauss (Progressão Aritmética).

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2)$$

Redução Geométrica de Escopo

Escopo do laço dividido por 2 a cada iteração. Contagem exata dominada pela PG Decrescente, derrubando a intuição quadrática.

$$\sum_{k=0}^{\log_2 n} \frac{n}{2^k} = 2n - 1 = \Theta(n)$$

Questão 4

SITUAÇÃO-PROBLEMA

Uma equipe de desenvolvimento está otimizando um algoritmo de compressão de imagens em tons de cinza para dispositivos de Internet das Coisas (IoT), que possuem restrições rígidas de consumo de energia e memória. Um dos submódulos críticos reduz sequencialmente a resolução de uma matriz de pixels de tamanho n (onde n é uma potência de 2) através de um laço de repetição aninhado. Para evitar estimativas vagas e intuitivas, os engenheiros precisam analisar matematicamente a contagem exata das instruções críticas executadas, garantindo que o tempo de resposta se mantenha dentro de limites estáveis e escaláveis para a infraestrutura do backend.

DADOS DE SUPORTE

De acordo com Cormen et al. (2012), a análise matemática de laços aninhados com dependências de controle exige a modelagem analítica da soma acumulada de suas iterações, expressa na forma de somatórios matemáticos ou séries geométricas decrescentes.

Considere o seguinte trecho de código-fonte estruturado em linguagem C em que n representa o tamanho do vetor de pixels de entrada:

```c
int processar_matriz(int n) {
    int count = 0;
    for (int i = n; i > 0; i /= 2) {
        for (int j = 0; j < i; j++) {
            count++; // Operacao critica de custo constante \Theta(1)
        }
    }
    return count;
}
```

Considere também a fórmula da soma de uma progressão geométrica (PG) finita de razão $q \neq 1$:

$$S_k = a_1 \cdot \frac{1 - q^k}{1 - q}$$

COMANDO DA QUESTÃO

A partir da análise do fluxo lógico de repetição do algoritmo e aplicando as propriedades matemáticas de séries geométricas, assinale a alternativa que apresenta a simplificação fechada exata do número de operações críticas realizadas e sua respectiva classificação assintótica de pior caso:

ALTERNATIVAS DE RESPOSTA

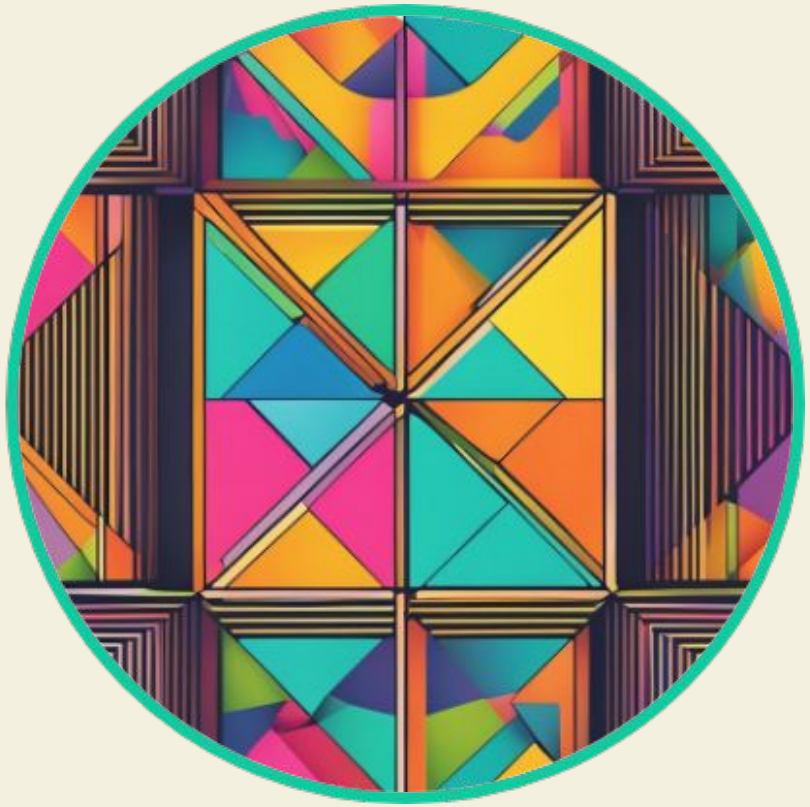
A) A contagem exata é expressa por $2n - 1$ operações críticas, estabelecendo uma classe de complexidade de tempo linear de ordem $\Theta(n)$.

B) A contagem exata é expressa por $n \log_2 n$ operações críticas, estabelecendo uma classe de complexidade de tempo linearítmica de ordem $\Theta(n \log n)$.

C) A contagem exata é expressa por $n(n + 1)/2$ operações críticas, estabelecendo uma classe de complexidade de tempo quadrática de ordem $\Theta(n^2)$.

D) A contagem exata é expressa por $2^n - 1$ operações críticas, estabelecendo uma classe de complexidade de tempo exponencial de ordem $\Theta(2^n)$.

Recursão e Teorema Mestre



Paradigma Divisão e Conquista

Resolve equações de recorrência comparando o custo de combinação com o limite de divisão.

- **Caso 1 (Recursão domina):** A complexidade é governada pelas folhas da árvore de chamadas.
- **Caso 2 (Equilíbrio):** Combinação e divisão empatam. O custo logarítmico é incorporado.
- **Caso 3 (Combinação domina):** O gargalo reside inteiramente na mesclagem dos subproblemas.

Questão 5

SITUAÇÃO-PROBLEMA

Uma equipe de engenharia de software de uma empresa de geoprocessamento está desenvolvendo um sistema de renderização de mapas de alta resolução. O módulo principal processa matrizes de elevação de terreno de dimensão $n \times n$ (onde n é uma potência de 2). Para otimizar o tempo de renderização, dois engenheiros seniores propuseram estratégias de projeto distintas baseadas no paradigma de divisão e conquista para realizar o processamento das matrizes. A escolha da estratégia correta requer uma análise crítico-reflexiva rigorosa para antecipar possíveis cenários de gargalo temporal em ambiente de produção sob cargas de trabalho extremas.

DADOS DE SUPORTE

De acordo com Cormen et al. (2012), a análise de algoritmos recursivos baseados em divisão e conquista fundamenta-se na modelagem de equações de recorrência do tipo $T(n) = aT(n/b) + f(n)$. A resolução analítica dessas recorrências pelo Teorema Mestre permite classificar e comparar a eficiência assintótica de diferentes abordagens.

Considere as duas estratégias de divisão e conquista propostas pela equipe de engenharia:

- **Estratégia Alfa (Algoritmo A):** Divide a matriz de dimensão $n \times n$ em 4 submatrizes quadradas de tamanho $n/2 \times n/2$. Cada submatriz é processada recursivamente e a combinação dos resultados é feita por uma varredura linear que consome tempo de execução de ordem $\Theta(n)$ nas bordas das submatrizes.
- **Estratégia Beta (Algoritmo B):** Divide o problema em apenas 2 subproblemas recursivos de tamanho $n/2$, mas exige um processamento de combinação quadrático que consome tempo de execução de ordem $\Theta(n^2)$ para correlacionar os elementos de suporte da entrada.

As equações de recorrência para os algoritmos A e B são dadas, respectivamente, por:

- $T_A(n) = 4T_A(n/2) + \Theta(n)$
- $T_B(n) = 2T_B(n/2) + \Theta(n^2)$

COMANDO DA QUESTÃO

A partir da modelagem matemática das equações de recorrência de ambos os algoritmos e aplicando as definições e os casos do Teorema Mestre, analise as afirmações sobre a eficiência assintótica das duas estratégias propostas e assinale a alternativa correta:

ALTERNATIVAS DE RESPOSTA

A) Ambas as estratégias apresentam a mesma complexidade assintótica de pior caso $T(n) = \Theta(n^2)$, indicando que os algoritmos possuem o mesmo comportamento de desempenho em escala, sendo indiferente a escolha técnica sob o ponto de vista de eficiência de hardware.

B) A Estratégia Alfa é assintoticamente superior à Estratégia Beta, pois a recorrência $T_A(n) = 4T_A(n/2) + \Theta(n)$ resulta em complexidade de tempo de execução $O(n \log n)$ pelo Caso 2 do Teorema Mestre, enquanto a recorrência $T_B(n) = 2T_B(n/2) + \Theta(n^2)$ resulta em complexidade quadrática $O(n^2)$.

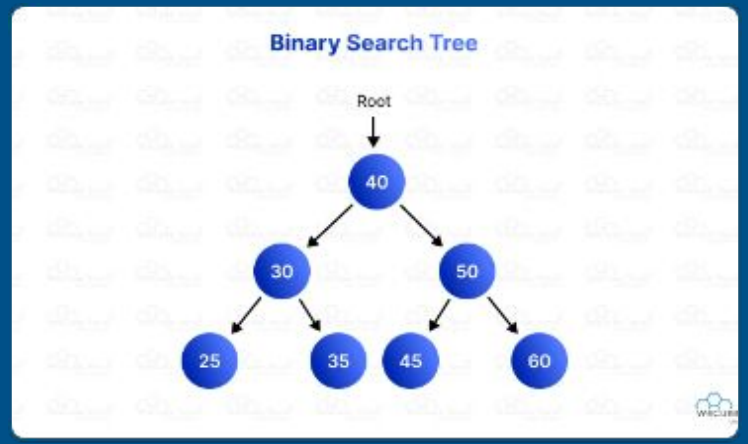
C) A Estratégia Beta é assintoticamente superior à Estratégia Alfa, uma vez que realiza apenas 2 chamadas recursivas, gerando uma equação de recorrência que resulta em complexidade logarítmica $O(\log n)$ pelo Caso 2 do Teorema Mestre, enquanto o Algoritmo A apresenta complexidade cúbica $O(n^3)$.

D) A Estratégia Alfa apresenta complexidade exponencial $T(n) = O(2^n)$ pelo Caso 1 do Teorema Mestre, enquanto a Estratégia Beta apresenta complexidade polinomial $T(n) = O(n^4)$, evidenciando a instabilidade crítica de laços recursivos que dependem de divisões de escopo fracionárias.

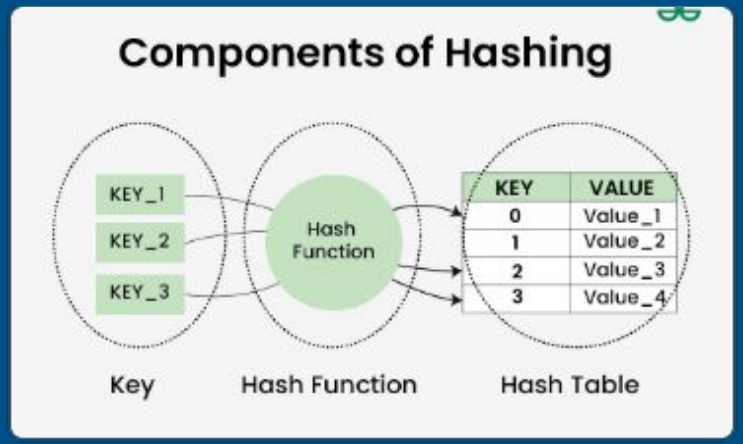
Trade-offs em Algoritmos de Ordenação

Algoritmo	Complexidade Garantida	Caso Médio (Prática)	Memória Auxiliar	Estabilidade
Merge Sort	Θ(n log n)	Θ(n log n)	Θ(n) Linear	Sim (Estável)
Quicksort	Θ(n^2) Pior Caso	**Excelente**	Θ(1) In-place	Instável
Heapsort	Θ(n log n)	Θ(n log n) (Baixa cache)	Θ(1) In-place	Instável

Estruturas de Dados Avançadas



Tempo médio Θ(log n). Se degenerar em inserção sequencial sem balanceamento, as buscas decaem para complexidade linear Θ(n).



Acesso em Θ(1). A degradação via alta colisão (encadeamento saturado) pode forçar buscas para Θ(n) absoluto.



Filtra top-k elementos em stream com custo restrito a Θ(n log k) e espaço Θ(k), evitando as inviáveis ordenações totais Θ (n log n).

Questão 6

SITUAÇÃO-PROBLEMA

Uma grande plataforma de comércio eletrônico precisa otimizar seu sistema de recomendação de produtos com base no histórico de cliques dos usuários em tempo real. O sistema processa um fluxo contínuo de n dados de interações. Para cada categoria de produto, o sistema deve manter uma coleção de dados de suporte contendo os k itens mais acessados (com $k \ll n$) para sugerir dinamicamente aos usuários. Para assegurar o cumprimento de acordos de nível de serviço (SLA) de baixíssima latência sob condições de pico, a equipe de desenvolvimento propôs diferentes arquiteturas de software baseadas no acoplamento de estruturas de dados avançadas e algoritmos de ordenação e busca.

DADOS DE SUPORTE

Cormen et al. (2012) destacam que o projeto de algoritmos e estruturas de dados de alta performance exige uma postura crítica para comparar trade-offs de tempo e espaço, refutando escolhas intuitivas ou coleções genéricas sem fundamentação analítica.

Considere as propostas concebidas pela equipe de desenvolvimento:

- **Abordagem I (Merge Sort & Vetor):** Acumula todas as n interações em um arranjo dinâmico contíguo e, a cada intervalo de tempo, aplica o algoritmo Merge Sort para reordenar o vetor e recuperar os k maiores elementos.
- **Abordagem II (Min-Heap de tamanho k):** Mantém um Heap Mínimo na memória com exatamente k elementos do fluxo. Cada novo elemento é comparado com o mínimo (raiz). Se for maior, a raiz é substituída por ele e a propriedade de Heap é restaurada via a função auxiliar `heapify`.
- **Abordagem III (Tabela Hash & Heapsort):** Insere todas as n chaves em uma Tabela Hash com resolução por encadeamento. Quando o painel solicita a exibição, extraem-se os elementos em um vetor e ordena-se o vetor via Heapsort para obter os k maiores.

COMANDO DA QUESTÃO

Considerando a aplicação prática dessas abordagens, os custos assintóticos associados aos algoritmos de ordenação e às estruturas de dados, assinale a alternativa que apresenta uma análise de projeto de software **INCORRETA**:

ALTERNATIVAS DE RESPOSTA

A) A Abordagem I é ineficiente para fluxos de dados de grande escala, pois demanda uma complexidade de tempo de pior caso de $O(n \log n)$ em cada reordenação periódica e exige espaço de memória auxiliar de ordem $O(n)$ devido ao uso do algoritmo Merge Sort.

B) A Abordagem II é a solução mais escalável sob a ótica de usabilidade e restrição de recursos, visto que o seu tempo de processamento total é limitado a $O(n \log k)$ e o consumo de memória adicional é restrito a $O(k)$ para manter o Heap Mínimo.

C) A Abordagem III apresenta excelente desempenho em tempo de pior caso, pois o Heapsort realiza a ordenação das chaves em tempo $O(n)$ sem consumir espaço de memória adicional, superando a latência de reorganização e inserção dinâmica gerada pelo Min-Heap.

D) O acoplamento de uma Tabela Hash na Abordagem III resulta em complexidade de tempo de pior caso de busca de $O(n)$ em caso de colisões críticas, violando os princípios de previsibilidade de latência e estabilidade exigidos para sistemas de alta performance.

Obrigado pela atenção!